Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Reti di Calcolatori

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Indice

1 Introduzione alle reti **4**
1.1 Comunicazione tra elaboratori 4
1.2 Commutazione di circuito e di pacchetto 7
1.3 Classificazione e misura dei ritardi 8
1.4 Modelli di riferimento ISO-OSI e TCP/IP 11
1.5 Indirizzi IP e subnetting 14
1.6 Esercizi svolti 16

2 Livello applicazione **17**
2.1 Il modello client/server 17
2.2 Hyper Text Transfer Protocol - HTTP 18
2.3 Domain Name Service - DNS 22
2.4 Posta elettronica - SMTP, POP3 e IMAP 24

3 Livello trasporto **26**
3.1 Scopi e servizi 26
3.2 Protocollo TCP 26
3.3 Protocollo UDP 26

4 Livello network **27**
4.1 Protocollo IP 27
4.2 Protocollo di configurazione DHCP 27
4.3 Protocollo ICMP 27
4.4 Protocolli di routing 27
4.5 IPv6 27
4.6 Network Address Translation - NAT 27

5 Livello data-link **28**
5.1 Framing 28
5.2 Accesso al canale condiviso 28
5.3 Sottolivello MAC 28

5.4 Bridge, switch e LAN 28
5.5 Wireless LAN 28

Capitolo 1

Introduzione alle reti

1.1 Comunicazione tra elaboratori

Il corso di reti di calcolatori si pone come obiettivo quello di risolvere il problema della **comunicazione tra computer**, inteso come il trasferimento di informazioni tra due o più sistemi, studiato in questo corso con un approccio **top-down**, ovvero a partire da un alto livello per andare sempre più a basso livello. In particolare, si presentano i due problemi fondamentali della comunicazione:

- **Protocollo di comunicazione**: Insieme di regole logiche che definiscono come le entità devono comunicare, nello specifico il **formato** dei messaggi, il loro **ordine** e le **azioni** da compiere alla loro ricezione.
- **Trasporto delle informazioni**: Il trasferimento fisico di dati fra due dispositivi. Richiede un'infrastruttura di rete composta da mezzi trasmissivi, come cavi o fibra ottica, dispositivi di rete e, naturalmente, protocolli di trasmissione.

Possiamo paragonare lo scambio di informazioni tra elaboratori con l'invio di una lettera. Il mittente scrive il messaggio, inserisce il foglio in una busta, con scritto l'indirizzo del destinatario e la consegna all'ufficio postale. Quest'ultimo la trasporterà fino al destinatario descritto, il quale potrà aprire la busta e leggere il contenuto del messaggio. Affinché il processo funzioni correttamente, è necessario seguire regole precise (quindi seguire il protocollo), come scrivere correttamente l'indirizzo del destinatario.

Nella pratica, la comunicazione tra due calcolatori avviene tramite una sequenza di livelli, spesso rappresentata come una pila verticale, dove ogni livello svolge una funzione specifica e prepara i dati per quello successivo. Intuitivamente:

1. L'applicazione genera il messaggio.
2. Il messaggio è preparato per la trasmissione.

3. I dati vengono inviati tramite la rete.
4. Il destinatario riceve i dati.
5. I dati sono elaborati fino a raggiungere l'applicazione finale.

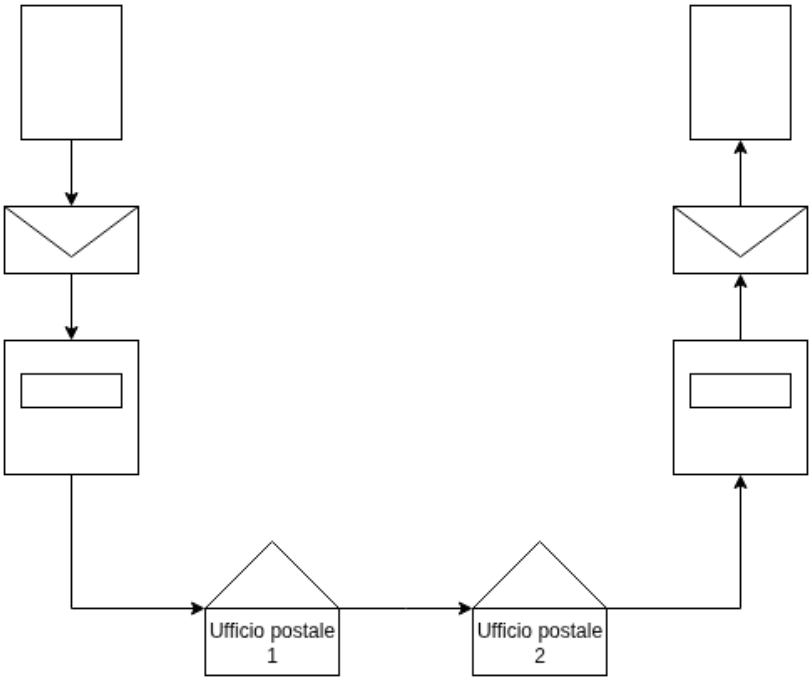


Figura 1.1: Esempio di comunicazione intuitivo

Naturalmente, lo scambio di informazioni non è unidirezionale, infatti si dice che il processo di trasferimento informazioni è **simmetrico** tra mittente e destinatario. Definiamo ora la struttura delle architetture di rete per capire in che modo comunichino gli elaboratori: possono avere elementi diversi, ma tutte presentano necessariamente gli **end-host**, ovvero i dispositivi per l'invio e ricezione di informazioni come computer o smartphone, gli **apparati di rete**, che permettono l'instradamento o la distribuzione dei dati nella rete, come router, switch e access point, e i **collegamenti**, i mezzi fisici o wireless che permettono la trasmissione dei dati, come fibra ottica o Wi-Fi.

Possiamo descrivere **Internet** in termini di infrastruttura di rete su scala globale, la quale fornisce servizi ad applicazioni distribuite. Per carpirne il funzionamento analizziamo un esempio su scala minore: le reti **Local Area Network** (LAN). Queste sono reti che coprono un'area limitata, come ad esempio un ufficio o una casa, dove sono presenti gli end-host. I principali dispositivi utilizzati sono:

- **Switch**: Dispositivo che collega fra loro più connessioni cablate.

- **Access point**: Dispositivo wireless che consente agli end-host di collegarsi alla rete.

Tipicamente lo switch rappresenta il punto centrale della rete cablata, mentre l'access point vi si collega per fornire connettività wireless. Nelle reti domestiche moderne è comune che router, switch e access point siano integrati nello stesso apparato.

Le LAN sono collegate alla rete Internet grazie ad un **router di bordo**, il quale, oltre al suddetto compito, è capace di inoltrare i dati ad altri router della rete globale, detta **backbone**, fino al raggiungimento di quello di destinazione, che consegnerà i dati all'end-host.

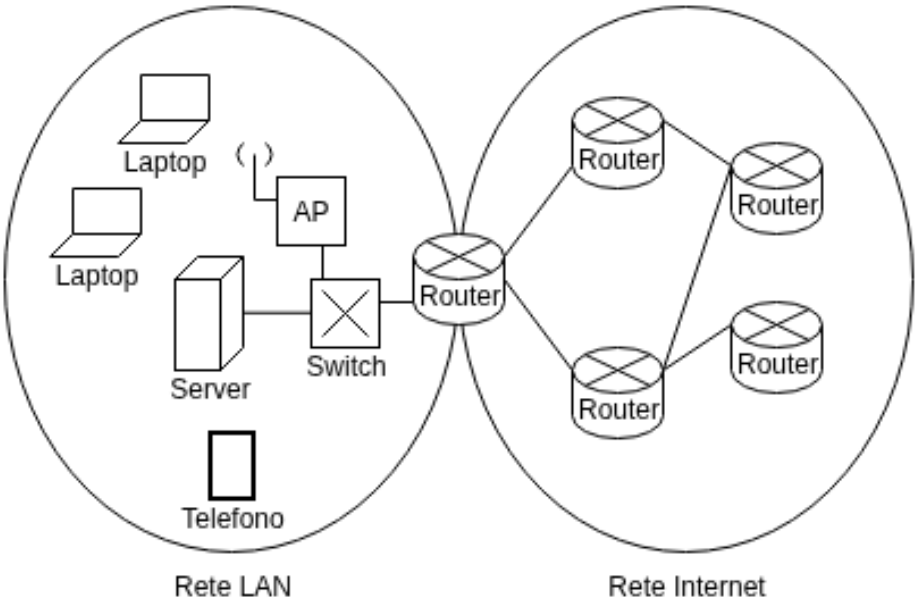


Figura 1.2: Collegamento tra rete LAN e Internet

La connettività Internet è fornita da organizzazioni chiamate **Internet Service Provider** (ISP), divise in tre livelli gerarchici:

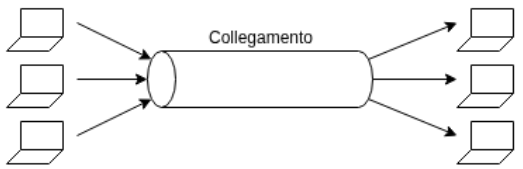
- **Tier 1**: Operatori globali che gestiscono infrastrutture backbone internazionali.
- **Tier 2**: Operatori nazionali o regionali che acquistano connettività dai Tier-1 ma possono anche stabilire collegamenti diretti tra loro.
- **Tier 3**: Operatori locali che forniscono connettività agli utenti finali, come abitazioni, aziende, hotel.

In particolare, esistono gli **Autonomous Systems** (AS); insiemi di reti gestite da un'unica entità amministrativa e con una politica di routing comune.

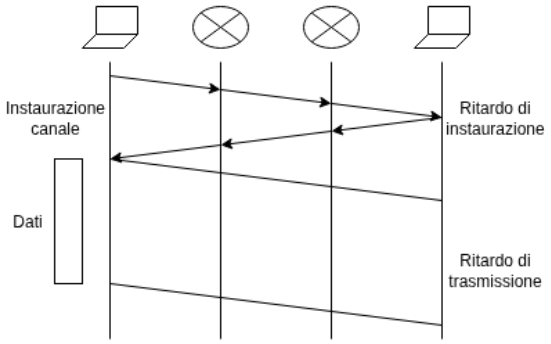
1.2 Commutazione di circuito e di pacchetto

Ci sono due modalità principali per il trasferimento dei dati fra calcolatori: la **commutazione di circuito** e **di pacchetto**. La prima è stata storicamente usata nelle reti telefoniche tradizionali e ad oggi è nettamente meno diffusa. Il percorso tra la sorgente e la destinazione è allocato staticamente per la durata dello scambio dell'informazione, vedendo quindi le risorse di trasmissione dedicate alla comunicazione. Tuttavia, essendo la banda limitata, si può sostenere solo un certo numero di circuiti allo stesso tempo.

La gestione delle risorse avviene tramite il **Time Division Multiplexing**, una divisione delle risorse in slot da fornire ai vari utenti. Alternativamente si utilizza il **Frequency Division Multiplexing**, una divisione di spazio sulle frequenze sicché gli slot non interferiscano gli uni con gli altri, seguendo la logica della radio.



(a) Commutazione di circuito



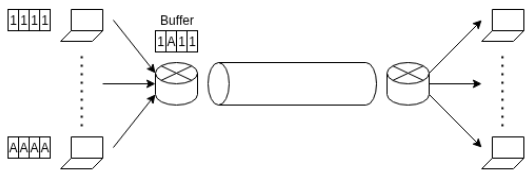
(b) Grafico di commutazione

Questo sistema di commutazione ha due pro: **risorse dedicate** a ciascun utente, creando una comunicazione senza interruzioni o accodamento, e il **ritardo prevedibile** e costante dopo l'instaurazione del circuito. I problemi riguardano in primo luogo la **banda limitata**, poiché dando slot dedicati, in caso fossero tutti occupati, non sarebbe possibile usare il servizio. In secondo luogo parliamo di spreco delle risorse, perché, fintanto che rimane aperto, il canale è mantenuto attivo anche quando non si trasferiscono i dati.

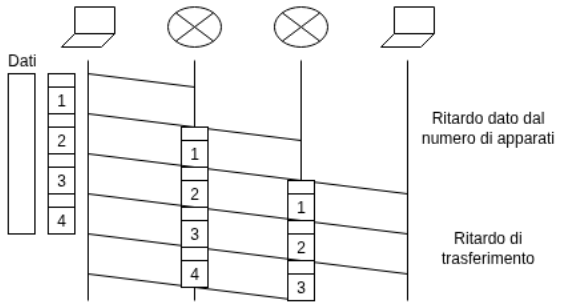
La **commutazione di pacchetto** invece vede i dati da trasferire divisi in sezioni indipendenti, a loro volta divise in due parti: l'**header**, contenente informazioni di controllo, tra cui indirizzo sorgente e destinazione, e la seconda, contenente i dati del pacchetto. Le varie sezioni sono inviate al router, il quale le accoda con un buffer per inviarle una alla volta passando per un bottleneck. Con questa metodologia è possibile gestire un numero molto elevato di utenti, poiché le risorse sono condivise dinamicamente. Infatti, non ci sono sprechi di risorse in caso un utente smetta di trasferire dati.

Il contro di questa modalità di commutazione è invece l'introduzione dei ritardi. I pacchetti si possono trasferire solo quando sono stati ricevuti tutti i relativi bit contenenti

le informazioni tramite la modalità **store-and-forward switching**[1]. In particolare, è presente un ritardo iniziale legato al numero di apparati attraversati; smaltito questo, ci sarà quello di trasmissione. Un secondo problema è dato dalla potenziale **perdita di pacchetti**, perché in quanto i buffer hanno spazio limitato, può capitare di richiedere il trasferimento di pacchetti quando questo è pieno. In tal caso, il pacchetto viene scartato.



(a) Commutazione di pacchetto



(b) Grafico di commutazione

(c) Divisione in pacchetti

1.3 Classificazione e misura dei ritardi

Le reti di calcolatori sono caratterizzate da un **throughput** limitato, inteso come la quantità di dati al secondo che può essere trasferita tra due sistemi; introducono ritardi e si possono perdere anche pacchetti. Sebbene questo non sia uno scenario ideale, possiamo analizzare i diversi tipi di ritardo e cercare di usare gli strumenti a disposizione per affrontare il problema. Generalmente, le prestazioni delle applicazioni per internet sono pesantemente influenzate dai ritardi di rete, somma dei ritardi introdotti in ciascun nodo, dati da ogni hop di router e composti da:

- **Ritardo di elaborazione**: Legato all'architettura del router, è il tempo impiegato per esaminare l'intestazione del pacchetto e determinare dove dirigerlo. In questo

[1] Informazione salvata nel router, poi inoltrata a quello di destinazione.

intervallo di tempo si possono anche controllare errori a livello di bit avvenuti nella trasmissione. Dopo l'elaborazione, si invia il pacchetto verso la coda che precede il collegamento al prossimo router.

- **Ritardo di accodamento**: Inserito in coda, il pacchetto deve attendere la trasmissione sul collegamento. Il tempo di attesa è legato al numero di pacchetti precedentemente arrivati. Infatti, nelle politiche first-come-first-served, comunemente in uso nelle reti a commutazione di pacchetto, il pacchetto può essere trasferito solamente dopo la trasmissione di tutti quelli che lo precedono nell'arrivo. Normalmente è il ritardo che incide più di tutti.

- **Ritardo di trasmissione**: Riguarda il tempo necessario per immettere tutti i bit del pacchetto nel canale di trasmissione. È un valore che dipende dalla sua dimensione e dalla velocità di trasmissione del router. In particolare:

  $$d_{trans} = L/R$$

  Con L dimensione del pacchetto in bit e R velocità di trasmissione in bit al secondo.

- **Ritardo di propagazione**: Immesso nel collegamento, questo è il tempo utilizzato per la propagazione di un bit fino al prossimo router. È quindi un valore legato dal mezzo fisico per la trasmissione e la distanza fra i due router. In particolare:

  $$d_{prop} = d/v$$

  Con d distanza tra i due router e v la velocità di propagazione nel collegamento.

Nella pratica, il ritardo complessivo dipende anche dal contesto geografico. Generalmente, valgono i seguenti valori:

- **Comunicazione su stessa LAN**: $< 1ms$
- **Comunicazione con stesso ISP**: $5 - 20ms$
- **Comunicazione su stesso continente**: $20 - 50ms$
- **Comunicazione intercontinentale**: $100 - 200ms$

Abbiamo poi a disposizione alcuni strumenti per la misurazione dei ritardi; il primo è il **ping**, un tool a CLI che permette di misurare il **Round Trip Time**, ovvero il tempo di andata e ritorno di un messaggio fra due elaboratori. Prende in input il nome logico utilizzato dal sito, come "www.google.com", e stampa a video indirizzo IP dell'host (approfondito in sezioni successive), minimo, media, massimo e deviazione standard del roundtrip time.

```
f27@F27-Laptop:~$ ping www.google.it
PING www.google.it (142.251.209.35) 56(84) bytes of data.
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=1 ttl=117 time=50.6 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=2 ttl=117 time=55.4 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=3 ttl=117 time=63.3 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=4 ttl=117 time=68.0 ms
64 bytes from tzmila-ae-in-f3.1e100.net (142.251.209.35): icmp_seq=5 ttl=117 time=61.9 ms
^C
--- www.google.it ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4004ms
rtt min/avg/max/mdev = 50.594/59.829/68.018/6.129 ms
```

Figura 1.5: Esempio di ping

Un altro strumento è il **traceroute**, il quale prende in input il nome dell'host come per il ping. Questo comando consente di vedere il percorso intrapreso dai pacchetti (tracciare il percorso, difatti) e determinare i ritardi di andata e ritorno per ogni singolo hop effettuato. In particolare, il sistema dell'utente invia un certo numero di pacchetti verso la destinazione e ad ogni router attraversato, questi rimandano un messaggio all'origine contenente il loro nome e indirizzo IP. Può capitare inoltre:

- **Router nascosto**: Se l'hop è indicato con tre asterischi, gli amministratori di rete hanno celato le sue informazioni.
- **Cammini equivalenti**: Se più indirizzi IP sono indicati nel passo, significa che gli hop sono equivalenti per raggiungere una stessa destinazione.

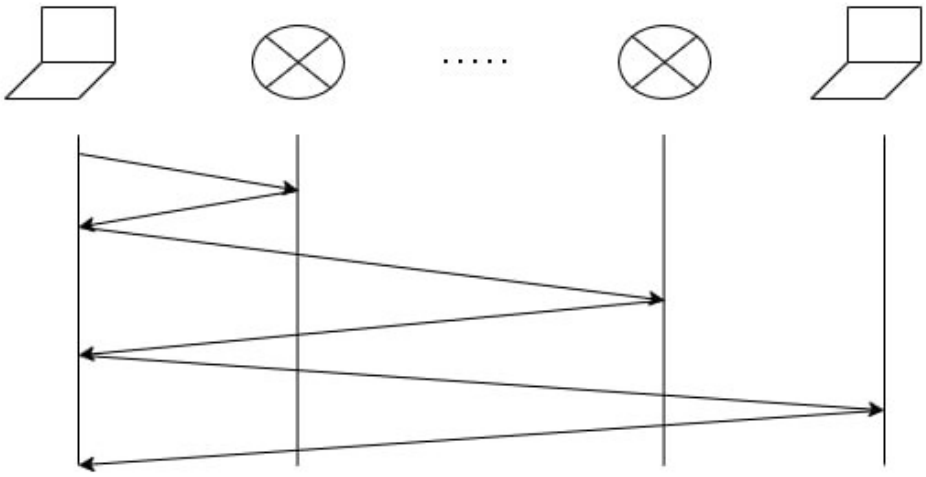


Figura 1.6: Visualizzazione di traceroute

Un'altra misura importante per le prestazioni di una rete è il **throughput end-to-end**. Si tratta, in ogni istante di tempo, della velocità alla quale un'entità sta ricevendo un determinato insieme di informazioni, come per esempio un file. Si tratta di un valore misurato in *bps* (bit per second). Generalmente si misura anche il **throughput medio**, dato dal rapporto fra il numero di bit F e il tempo T impiegato per la loro intera ricezione: $\overline{R} = F/T$.

Capiamo dunque che è un valore dipendente dalla velocità di trasmissione. Non a caso, essendo i link collegati tramite dei **bottleneck**, la velocità di trasferimento è determinata dal link con capacità minima, risultando anche quello più lento.

1.4 Modelli di riferimento ISO-OSI e TCP/IP

La modalità di trasferimento delle informazioni è organizzata in base al **modello a strati**. Il problema della comunicazione fra due elaboratori ha una parte che riguarda la logica, quindi la gestione delle informazioni, e una che riguarda la fisica, ovvero la strutturazione dell'architettura di rete. L'approccio utilizzato per la risoluzione del primo problema è dato dal **Divide et Impera**: intuitivamente, date due entità comunicanti, la prima "impacchetta" il messaggio aggiungendo un header per ogni strato, e la seconda lo "spacchetta" rimuovendone uno per livello per poterlo leggere.

In un modello simile potremmo aggiungere un numero indefinito di livelli il cui insieme è detto **stack protocollare** ed è imperativo che vengano eseguiti con un ordine specifico, perché se così non fosse, il messaggio potrebbe non essere compreso.

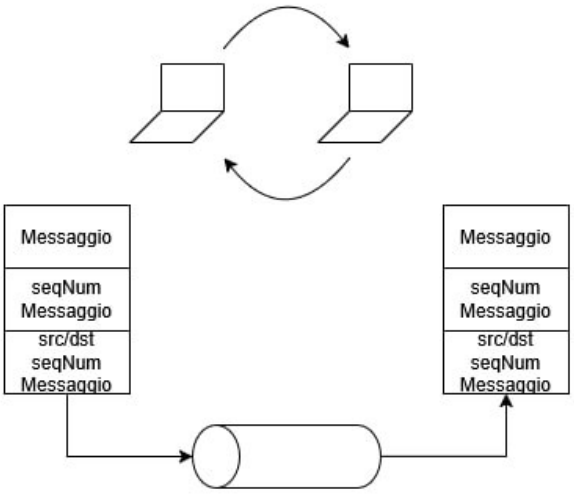


Figura 1.7: Modello a strati

Per esempio, se un calcolatore volesse inviare un messaggio, si potrebbe aggiungere prima un header per salvare il numero di sequenza del pacchetto. Essendo questi inviati non

necessariamente con un ordine fisso, risulterebbe utile conoscere l'ordine per ricomporli. In uno strato inferiore poi si potrebbero aggiungere i dati per indicare l'origine e la destinazione del messaggio. Inviando questo prodotto al secondo calcolatore, inizierebbe col rimuovere l'intestazione degli indirizzi, poi quella dei numeri di sequenza, per poi leggere il messaggio ordinato.

Un primo modello che utilizza questa struttura è l'**ISO/OSI**; proposto dalla International Organization for Standardization, per la Open System Interconnection. Si tratta del primo riferimento per la descrizione delle operazioni eseguite in ogni livello.

Il modello identifica due entità: gli **end-system**, i calcolatori, e gli **intermediate-systems**, i router. Entrambe condividono i primi tre livelli a partire dalla base: quello **fisico** per la gestione dei bit, quello di **collegamento dati** per determinare la destinazione e quello di **rete**, responsabile dell'instradamento. Gli end-system hanno inoltre quattro livelli aggiuntivi: **trasporto**, **sessione**, **presentazione** e **applicazione**.

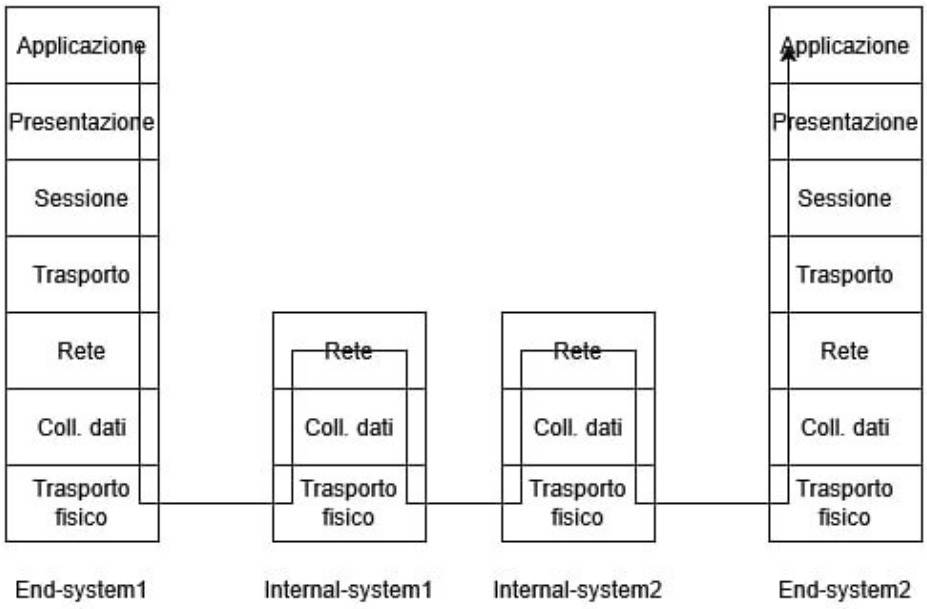


Figura 1.8: Modello ISO/OSI

Nella pratica si usa un modello che usa questo come base, ma presenta solo i livelli aggiuntivi di trasporto e applicazione. Si chiama **modello TCP/IP** ed il nome deriva dai due principali protocolli usati in questi due livelli: **Transmission Control Protocol** e **Internet Protocol**. Analizziamone ogni strato:

- **Livello di applicazione**: Sede delle applicazioni di rete e dei relativi protocolli come HTTP (scambio documenti web), SMTP (scambio mail), FTP (scambio

file tra due sistemi remoti). Generalmente scambia pacchetti di informazioni, i **messaggi**, con l'applicazione in un altro sistema periferico.

- **Livello di trasporto**: Si occupa del trasferimento dei messaggi tra punti periferici gestiti dalle applicazioni. Si usa il protocollo **TCP**, che fornisce un servizio orientato alla connessione, quindi che permette di stabilirla, usarla e rilasciarla. Include inoltre la garanzia di consegna dei messaggi a livello applicazione e un controllo di flusso, ottenuto frazionando i messaggi lunghi in più **segmenti**.

- **Livello di rete**: Si occupa di trasferire i pacchetti a livello di rete, detti **datagrammi**, da un host a un altro. Mette a disposizione il servizio di consegna segmento al livello di trasporto nell'host ricettivo. Comprende in primo luogo il protocollo **IP**, che definisce i campi dei datagrammi e come i sistemi agiscono su di essi. In secondo luogo ha vari protocolli di instradamento che determinano i percorsi da intraprendere.

- **Livello di collegamento**: Fornisce servizi per il trasferimento dei datagrammi fra i nodi del percorso, i quali dipendono dal protocollo utilizzato. È verosimile che questi vengano gestiti da protocolli diversi come Wi-Fi o Ethernet. A questo livello i pacchetti sono chiamati **frame**.

- **Livello fisico**: Si occupa del trasporto dei singoli bit dei frame da un nodo a quello successivo. Le modalità dipendono dal protocollo attuato, ma anche dal mezzo trasmissivo, come fibra ottica o doppino.

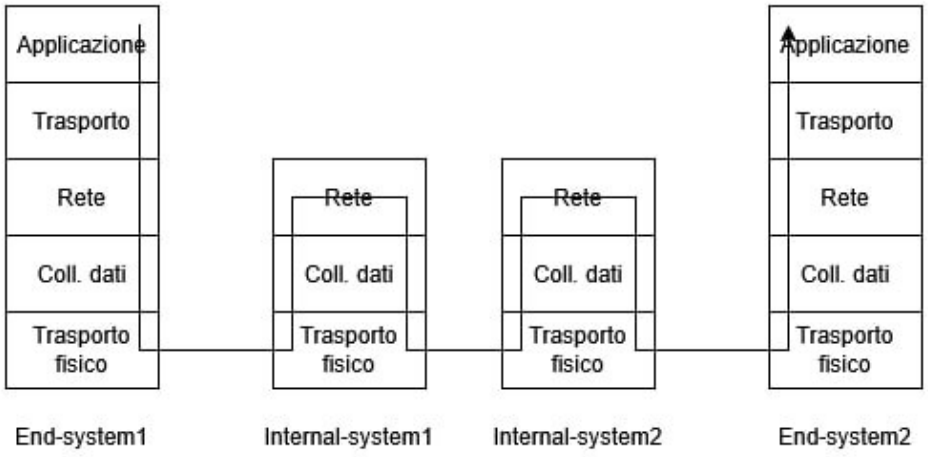


Figura 1.9: Modello TCP/IP

Questo processo che attraversa ogni strato è detto **incapsulamento**. Ora che conosciamo il senso degli strati, procediamo a definire le entità coinvolte nella comunicazione: a

livello applicativo, questa avviene tra processi in esecuzione sugli end-host. Affinché due processi possano comunicare, è necessario identificarli univocamente.

Un processo è identificato tramite un **numero di porta**, definito a livello di trasporto, mentre l'host sul quale è in esecuzione si identifica con un **indirizzo IP**. L'insieme di indirizzo IP e numero di porta costituisce un **endpoint di comunicazione**. Formalmente, una comunicazione tra due processi è identificata dalla seguente tupla:

$$(\mathrm{IP}_A, \mathrm{IP}_B, \mathrm{porta}_A, \mathrm{porta}_B)$$

1.5 Indirizzi IP e subnetting

Gli **indirizzi IP**, o indirizzi di protocollo internet, sono identificativi univoci di un'interfaccia di rete di un host o router all'interno della rete Internet. Sebbene si tenda ad associare questo codice di quattro numeri ad una sola macchina, è possibile, tramite alcune strategie, usarne uno diverso.

Naturalmente, i siti web non sono letti con sequenze di numeri dalle persone, ragion per cui presentano un grado di astrazione dato dal **nome di dominio**, come per esempio "www.google.com", al quale è associato l'indirizzo IP. La traduzione fra i due avviene a livello di applicazione tramite il **sistema DNS** (Domain Name System), il cui prodotto sarà usato a livello rete per aggiungere un'intestazione contenente l'indirizzo di sorgente e quello di destinazione. In base alla versione del protocollo abbiamo diversi bit totali per contenerli: $32b$ per **IPv4** e $128b$ per **IPv6**.

Gli indirizzi IP sono letti come sequenze di numeri in base binaria. Per esempio, con protocollo IPv4, il sistema potrebbe leggere:

$$1001\ 1101\ 0001\ 1011\ 0001\ 0011\ 0111\ 1011$$

Per una scrittura più contratta si utilizza la **notazione decimale puntata**, dove si considerano blocchi di $8b$ e si fa il cambio di base in decimale. Usando i numeri appena visti otteniamo:

$$\begin{aligned} 10011101 &= 157 \\ 00011011 &= 27 \\ 00010011 &= 19 \\ 01111011 &= 123 \end{aligned} \tag{1.1}$$

Dunque il nostro indirizzo IP si annoterà con 157.27.19.123. Da questi valori è possibile inoltre ricavare alcune utili informazioni. Non dissimilmente dai numeri telefonici per le reti fisse, ogni numero ha un significato. Prendendo come esempio +39 045 802 7059, sappiamo che +39 indica l'area geografica internazionale, 045 la provincia di Verona, 802

l'organizzazione specifica alla quale è associato il numero, e 7059 il suffisso, che indica il numero di interfaccia in uso.

La stessa logica vale per gli indirizzi IP: in questo esempio i primi tre blocchi sono l'identificativo di una rete specifica e il loro insieme è noto come **prefisso**, mentre l'ultimo blocco è detto **suffisso** e indica l'interfaccia dell'host. Ciò significa che possiamo ricondurci ad un dispositivo preciso in base al suo IP.

Nella notazione puntata, è possibile che un'organizzazione abbia un elevato numero di dispositivi, rendendo fuorviante la comprensione del suffisso. Per eludere questo problema, si indica il numero di bit che compongono il prefisso con una barra: 157.27.19.123/16. In particolare, esistono dei valori particolari riservati ad utilizzi specifici e non per l'identificazione dell'host:

- **This Host**: Un indirizzo in cui ogni bit è posto a zero, ovvero 0.0.0.0. Può essere anche usato per gli indirizzi non assegnati.
- **Local Broadcast**: Un indirizzo in cui ogni bit è posto a uno. Per il protocollo IPv4 vale 255.255.255.255.
- **Indirizzo di rete**: Un indirizzo il cui suffisso ha tutti i bit posti a zero, mentre la rete è indicata dal prefisso, come 127.27.0.0.
- **Directed Broadcast**: Un indirizzo in cui il suffisso ha tutti i bit posti a uno, mentre la rete è indicata dal prefisso, come 127.27.255.255.

Per identificare gli host disponibili in una determinata rete, i calcolatori utilizzano la **netmask**; una sequenza di $32b$ associata ad un indirizzo IP in cui i bit del prefisso sono posti a uno, mentre gli altri a zero. Possiamo annotarla, come già visto, in notazione decimale puntata o esadecimale. Sulla maschera, i calcolatori effettuano un'operazione di AND bit a bit con l'indirizzo IP, ottenendo l'indirizzo di rete.

È possibile verificare i valori di indirizzo IP e netmask con un comando a linea da terminale; per i sistemi UNIX è chiamato **ifconfig**[2], ma se si vuole andare più a fondo, usando il comando **whois** seguito da un indirizzo IP, è possibile ottenere informazioni sull'assegnazione degli indirizzi, il nome della rete e anche l'ISP al quale è allacciata.

Tra l'altro, questi due comandi possono presentare valori il cui prefisso ha un numero di bit diverso; ciò può accadere a causa della scelta architetturale di **subnetting**: la suddivisione di una rete principale in varie sottoreti, le quali avranno indirizzi ad esse collegate. Supponiamo infatti di voler dividere una rete principale di indirizzo 157.27.0.0/16 in due sottoreti di dimensione equivalente. Si ritiene necessario avere un modo per distinguere a quale delle due siamo connessi. A partire dall'indirizzo IP, attacchiamo un bit del suffisso al prefisso per usarlo come discriminante. Ciò ci fa ottenere:

[2]Il comando è stato deprecato di recente. Consigliabile usare **ip addr**.

- subNet1: prefisso(1011 1101 0001 1011 0), suffisso(000 0000 0000), notazione decimale: 157.27.0.0/17
- subNet2: prefisso(1101 1101 0001 1011 1), suffisso(000 0000 0000), notazione decimale: 157.27.128.0/17

Questa scelta riduce il numero di host disponibili per ciascuna sottorete, ma consente una gestione più efficiente della rete. Inoltre, questa struttura è impercettibile dall'esterno, in quanto i router di bordo delle sottoreti si connettono a quello della rete intera.

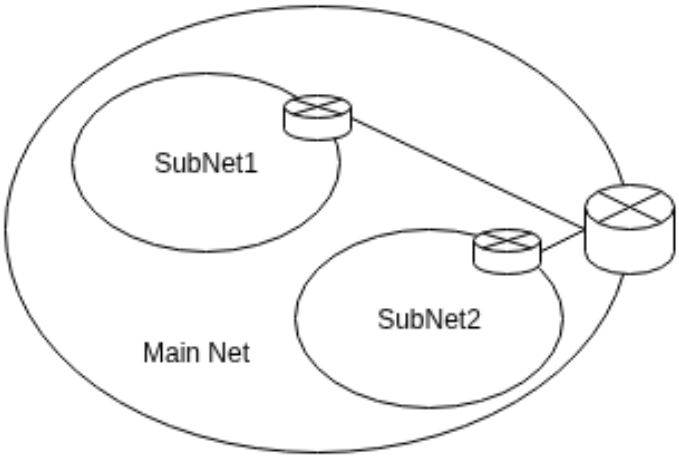


Figura 1.10: Subnetting

I blocchi di indirizzi sono detti **CIDR**: Classless Inter-Domain Router, sui quali è necessario aprire una parentesi storica. In precedenza, infatti, si chiamavano Classful ed essendo al tempo un servizio non ancora utilizzato su larga scala, ci si poteva permettere di "sprecare" disponibilità. Il numero di bit dedicati al prefisso era infatti predeterminato e codificato nello stesso; veniva usato per distinguere le diverse classi. In particolare:

- **Classe A**: L'indirizzo ha 0 come prima cifra e il prefisso ha $8b$ di dimensione. Numero di reti: 2^7. Numero di host per rete massimi: 2^{24}.
- **Classe B**: L'indirizzo ha 10 come prime due cifre e il prefisso ha $16b$ di dimensione. Numero di reti: 2^{14}. Numero di host per rete massimi: 2^{16}.
- **Classe C**: L'indirizzo ha 110 come prime tre cifre ed il prefisso ha $24b$ di dimensione. Numero di reti: 2^{21}. Numero di host per rete massimi: 2^8.

1.6 Esercizi svolti

Capitolo 2

Livello applicazione

2.1 Il modello client/server

Al livello applicativo sono generati i messaggi da scambiare tra i calcolatori, come per esempio richieste e risposte HTTP, un protocollo associato ai browser web, messaggi di posta elettronica, o porzioni di video. I messaggi generati dall'applicazione vengono passati al livello di trasporto, che può suddividerli in segmenti e aggiungere intestazioni contenenti informazioni di controllo come il numero di sequenza. Le sezioni devono poi selezionare il servizio del livello di trasporto, tra:

- **Protocollo TCP**: Servizio orientato alla connessione il quale garantisce la consegna dell'informazione.
- **Protocollo UDP**: Servizio non orientato alla connessione il quale non garantisce la consegna dell'informazione.

Applicazioni diverse hanno necessità di servizi diversi. Per esempio, un client di posta elettronica richiede che ogni informazione sia consegnata al destinatario; di conseguenza userà un protocollo che si appoggia alla garanzia di consegna del TCP, come **SMTP**.

Invece, un sito web come Twitch fornisce un contenuto "Tollerante alle perdite", quindi se non tutti i dati sono consegnati non necessariamente l'esperienza sarà danneggiata. Per esempio durante una livestream è richiesto di mantenere una connessione in tempo reale, privilegiando la continuità della riproduzione rispetto alla completa affidabilità dei dati, accettando eventuali ritardi o perdite limitate.

La "selezione del servizio" per l'invio di un messaggio si traduce in termini di apertura di un **socket** verso la destinazione, ovvero un'astrazione software che rappresenta il punto di accesso tra applicazione e livello di trasporto. Per aprirne uno è necessario specificare **protocollo** di trasporto, **indirizzi IP** di sorgente e destinazione e **porta** di sorgente e destinazione, le quali identificano l'interfaccia. I due processi che comunicano con il socket si identificano nei ruoli di:

- **Client**: Processo che inizia la comunicazione.
- **Server**: Il processo in attesa di contatto; accetta le richieste di comunicazione.

È un errore comune quello di confondere l'host con il client. Sebbene spesso un host coincida con un singolo client o server, una macchina può eseguire contemporaneamente più processi, assumendo quindi entrambi i ruoli.

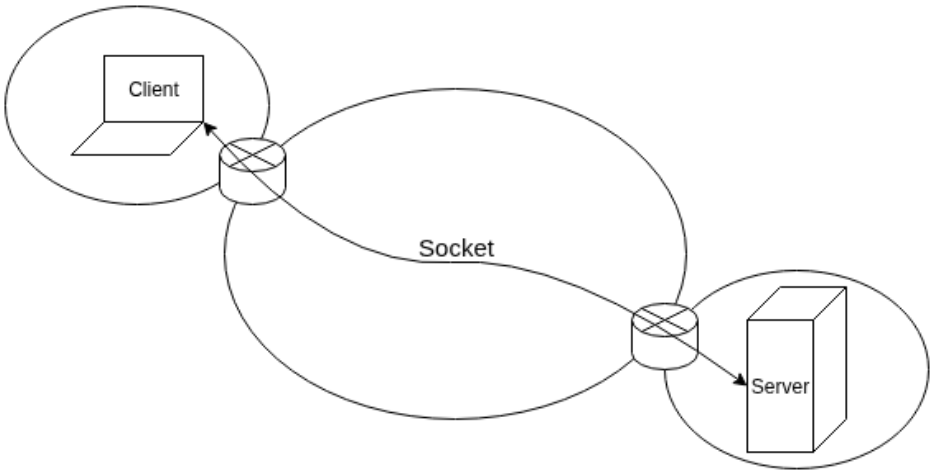


Figura 2.1: Apertura di un socket

2.2 Hyper Text Transfer Protocol - HTTP

HTTP è un protocollo a livello di applicazione del Web implementato in due processi client e server, i quali comunicano mediante uno scambio di messaggi HTTP. Il protocollo definisce struttura e modalità di scambio dei messaggi e si serve di **pagine Web**, generalmente costituite da un file HTML (Hypet Text Markup Language) principale detto **index.html** entro il quale si referenziano gli **oggetti**; file indirizzabili tramite URL come immagini, file JS o un foglio di stile CSS.

Il lato client del protocollo è implementato da un **browser Web**, come Firefox, per l'invio delle richieste, mentre il server è gestito dal **Web server**, il quale ospita gli oggetti menzionati prima. Quando un utente richiede una pagina Web, il browser invia al server messaggi di **richiesta HTTP** per ottenere gli oggetti nella pagina; si riceve quindi una **risposta HTTP**, la quale può essere positiva, confermando che il trasferimento degli oggetti è andato a buon fine, oppure negativa se la pagina non è stata trovata o è inesistente.

HTTP si appoggia al protocollo di trasporto TCP; infatti, il client stabilisce in primo luogo una connessione TCP con il server, alla quale i processi accedono tramite le proprie

socket. Inoltre, il server invia i file richiesti senza memorizzare alcuna informazione di stato del client; per questa ragione diciamo che HTTP è un protocollo **senza memoria di stato**.

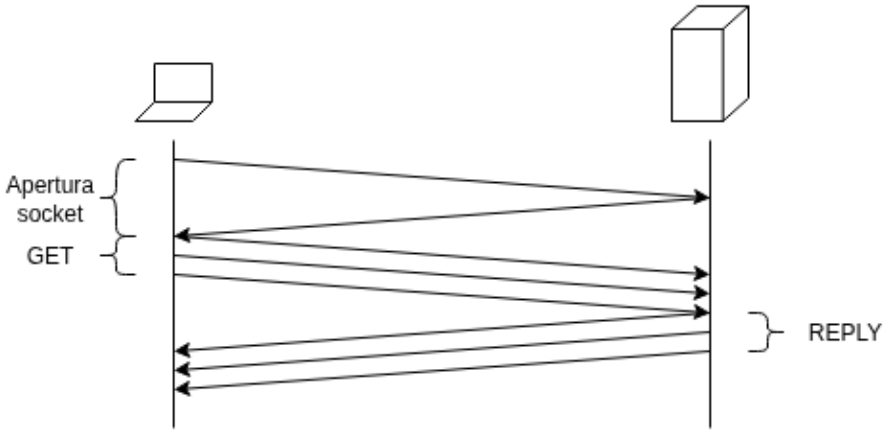


Figura 2.2: Protocollo HTTP

Presentiamo ora la struttura dei messaggi di richiesta e risposta HTTP. Il primo è composto da più righe: la prima è detta **riga di richiesta** e presenta tre campi:

- **Metodo**: Indica la procedura da attuare, come GET per richiedere risorse, POST per inviare informazioni al server, o DELETE, per cancellare informazioni sul server.
- **URL**: Identifica gli oggetti sui quali usare il metodo.
- **Versione HTTP**: Indica la versione del protocollo utilizzata.

Le righe successive sono dette di **intestazione** e tutte tranne quella che indica l'host (nella versione 1.1) sono opzionali. Determinano dati diversi come il browser utilizzato o la lingua del messaggio da inviare, come anche il tipo di connessione. Segue esempio:

```
GET /index.html HTTP/1.1
Host: www.univr.it
Connection: close
User-agent: Mozilla/4.0
Accept-language: it
```

I messaggi di risposta HTTP presentano invece tre sezioni: una **riga di stato** iniziale, delle **righe di intestazione** e il **corpo**, contenente la risorsa richiesta.

La prima riga è composta a sua volta da tre campi: la versione del protocollo, il codice di stato e il suo corrispettivo messaggio di stato, come [200-OK], [400-Bad request] o [404-Not found]. Le successive righe invece indicano molte informazioni come, tra l'altro, il tipo di connessione, la data e l'ora di ottenimento e invio della risorsa, il nome del web server e la data e l'ora dell'ultima modifica della risorsa. Segue esempio:

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2020 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2020 15:11:03 GMT
Content-Length: 3164
Content-Type: text/html
(Contenuto della risorsa...)
```

Nella creazione delle applicazioni web gli sviluppatori devono fare una scelta: inviare le coppie richiesta/risposta su connessioni TCP separate o sulla stessa? Il primo approccio si realizza con **connessioni non persistenti**, mentre il secondo con le **persistenti**. Sono entrambe scelte valide e funzionali, sebbene di default, HTTP implementi la seconda modalità.

Supponiamo di avere una connessione non persistente e che un client richieda le risorse di una pagina web con 10 immagini JPEG. In sequenza, avviene:

1. Client HTTP inizializza una connessione TCP con il server sulla porta 80, default per HTTP. Associate alla connesione ci sono due socket: uno per il client, l'altro per il server.

2. Il Client HTTP invia tramite il socket una richiesta HTTP al server.

3. Il Server HTTP riceve il messaggio del client, recupera l'oggetto dalla propria memoria e lo incapsula in una risposta HTTP per inviargliela tramite il socket.

4. Il Server HTTP comunica a TCP di terminare la connessione, cosa che avviene una volta consegnato il messaggio intero al client.

5. Il Client riceve la risposta, estrae il file dal messaggio e lo esamina, trovando i riferimenti alle immagini JPEG.

6. I primi quattro passi sono ripetuti per ogni oggetto da recuperare.

Essendo che si apre e chiude una connessione TCP per ogni risorsa da recuperare, avremo un totale di 11 connessioni. Queste possono essere settate per lavorare sequenzialmente o in parallelo.

A partire da HTTP 1.1 sono state introdotte le connessioni persistenti, dove il server mantiene la connessione TCP aperta anche dopo l'invio di tutte le risorse. Si possono inviare intere pagine web in una volta, come anche più pagine web, allo stesso client. La connessione ha termine quando rimane inattiva per un dato lasso di tempo configurabile.

Come già menzionato, i server HTTP sono privi di stato, per rendere lo sviluppo di applicazioni più semplice e con prestazioni migliori. Tuttavia, accade spesso di dover autenticare gli utenti, per ragioni di sicurezza e per fornire contenuti in funzione della loro identità. I **cookies** servono esattamente a questi scopi: consentono ai server di tener conto degli utenti.

Al primo utilizzo della pagina web, il server crea un codice associato all'utente che verrà inviato nella reply HTTP con l'header **SET COOKIE**. Il client, se accetta, memorizzerà l'indirizzo web e lo associerà al codice ricevuto. Nelle successive interazioni, se i cookie sono stati accettati, all'invio della GET da parte del client sarà presente anche un campo con il codice del cookie, il quale verrà elaborato dal server per fornire una risposta ritagliata per l'utente, a prescindere dall'indirizzo IP con il quale si collega.

Un'altra importante entità di rete è la **web cache**, nota anche come proxy server. Si pone nel percorso fra client e web server e soddisfa richieste HTTP al posto di quest'ultimo con lo scopo di aumentare le prestazioni. Diminuisce infatti il ritardo di accesso alle risorse più utilizzate e di consegna dei messaggi; come conseguenza diretta, il carico di rete è nettamente diminuito.

Senza web cache, avremmo una situazione nella quale più utenti in una rete richiedono una stessa risorsa, più si sperimenterà un ritardo. Se invece l'architettura di rete la comprende, si presenterebbe il seguente scenario:

1. Browser stabilisce una connessione TCP col proxy server e invia una richiesta HTTP per la risorsa.

2. Il proxy controlla la presenza di una copia della risorsa all'interno della propria memoria. Se è rilevata, la ritorna al client, altrimenti si apre una connessione TCP con il web server di origine per ottenerla.

3. Quando il proxy riceve l'oggetto, ne salva una copia in memoria e ne invia un'altra al browser web.

In genere le cache riescono a soddisfare il 70-80% delle richieste, ma usando un server diverso da quello originale si pone il problema dell'attualizzazione dei dati. Se una risorsa è modificata nel server originale, questa non è automaticamente inviata al proxy. La soluzione alla questione si presenta nella forma del **get condizionale**: una richiesta get-HTTP con una riga di intestazione **if-modified-since**. Fondamentalmente, il browser effettua la richiesta specificando una data, ed è ricevuta dal proxy. Questo comunicherà

con il web server che ritornerà la risorsa se è stata modificata entro la data specificata all'inizio. Se non ha subito modifiche, il proxy può inviarlo direttamente, altrimenti la risorsa deve essere ripresa dal web server originale.

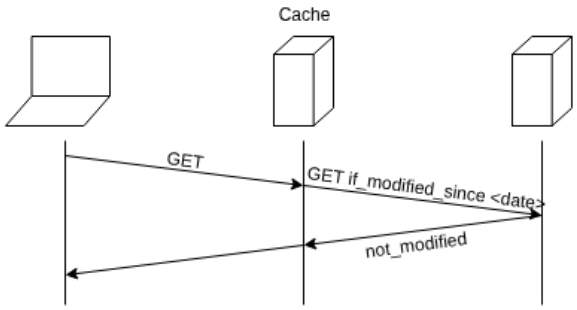


Figura 2.3: Web caching

2.3 Domain Name Service - DNS

Il servizio di directory di Internet **Domain Name Service** (DNS) ha lo scopo primario di tradurre i nomi di dominio associati ad un indirizzo IP per ritornare quest'ultimo e renderlo leggibile ai calcolatori. Si tratta in primo luogo di un database di server distribuito a livello mondiale implementato con una logica gerarchica e in secundis è un protocollo a livello di applicazione che consente agli host di interrogare la base di dati. In particolare, poggia su UDP e usa la porta 53; viene spesso utilizzato da altri protocolli a livello di applicazione, come HTTP e SMTP.

Supponiamo adesso che un web browser richieda la traduzione del nome di dominio "www.google.it". Il funzionamento del DNS segue le seguenti fasi:

1. Il calcolatore dell'utente esegue il lato client dell'applicazione DNS.
2. Il browser estrae il nome dell'host dall'URL e lo passa al lato client dell'applicazione DNS.
3. Il client DNS invia una **DNS query** contenente questo nome al server DNS.
4. Il server DNS ritorna l'indirizzo IP corrispondente al dominio ricevuto.
5. Ricevuto l'indirizzo IP, il browser web può adesso instaurare una connessione TCP verso il processo server collegato alla porta 80 di quell'indirizzo IP.

Come si può intuire, questa procedura introduce un ritardo, ma è comune che l'indirizzo IP desiderato si trovi all'interno di server DNS cache vicini, con lo scopo di ridurre il traffico in rete e il ritardo del servizio.

Il DNS offre anche altri servizi, come quello di **host** e **mail aliasing**, la possibilità di usare un insieme di nomi logici dal valore equivalente (sinonimi) per ricondursi ad uno stesso host, e la **distribuzione del carico di rete**, perché è possibile replicare su più web server uno stesso nome di dominio particolarmente richiesto, al quale è associato un insieme di indirizzi IP; quando richiesto dalla query, il DNS server risponderà con l'intero insieme di indirizzi variando l'ordinamento a ogni risposta.

Come già detto, più server DNS sono distribuiti per il mondo attraverso un **sistema gerarchico**; supponendo che un web browser effettui una DNS query, si esplorerebbero nel seguente ordine le tipologie dei server DNS:

1. **Server radice**: Contengono un insieme di indirizzi dei server TLD.
2. **Server Top-level domain**: Si occupano dei domini di primo livello come com, it, org e simili. Forniscono gli indirizzi IP dei server autoritativi.
3. **Server DNS autoritativi**: Appartenenti alle organizzazioni, specificano il loro dominio, come google, univr e simili.

Infine, le organizzazioni posseggono dei **server DNS locali**, i cui indirizzi IP si possono ottenere tramite una richiesta di protocollo DHCP.

La distribuzione dei DNS server risolve in particolare il problema del sovraccarico, distribuendo il traffico di query, e quello della disponibilità del servizio, poiché in caso di mancata risposta da parte di un server è possibile interrogarne un altro dello stesso livello.

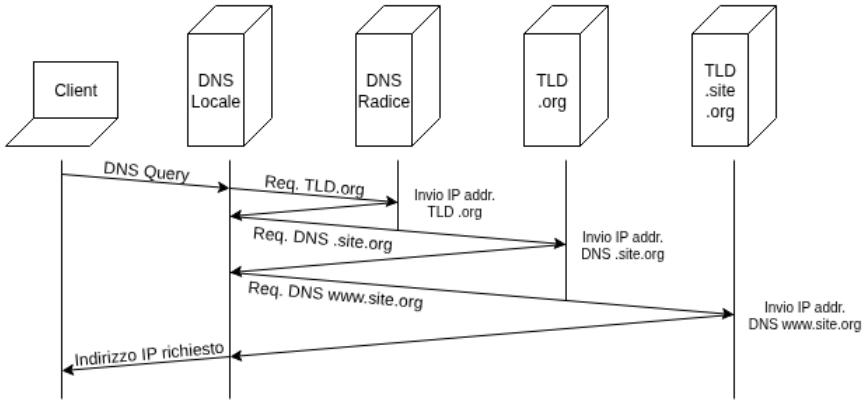


Figura 2.4: Ricerca del DNS

2.4 Posta elettronica - SMTP e IMAP

La posta elettronica è un mezzo di comunicazione asincrono che consente agli utenti di inviare e ricevere messaggi testuali; le e-mail moderne presentano inoltre altri oggetti, come **allegati**, **collegamenti ipertestuali**, **foto incorporate** e testo con **formattazione HTML**. Il meccanismo funziona grazie a tre elementi principali:

- **User agent**: Conosciuti anche come mail client, consentono di leggere, rispondere, inoltrare, salvare e comporre i messaggi. Esempi pratici sono Outlook, Gmail o Tutamail.
- **Mail server**: Costituiscono la parte centrale dell'infrastruttura del servizio; una casella di posta contiene i messaggi ricevuti alla quale è possibile accedere tramite autenticazione. Presenta inoltre una coda dei messaggi dove si trattengono quelli non ancora inviati.
- **Protocollo SMTP**: Il principale protocollo a livello di applicazione su Internet; si affida al TCP per trasferire i messaggi.

Lo scambio di messaggi tra due utenti vede generalmente una comunicazione diretta tra i due mail server; ciò significa che non sono utilizzati server intermedi, e se quello del destinatario è spento, si riproverà a inviare la mail. In condizioni ideali, invece, segue sei fasi:

1. L'utente A invoca il proprio user agent e fornisce l'indirizzo di posta elettronica dell'utente B. Compone il messaggio e dà istruzione allo user agent di inviarlo.
2. Lo user agent dell'utente A invia il messaggio al suo mail server, dove è collocato in una coda di messaggi.
3. Il lato client di SMTP, eseguito sul server dell'utente A, vede il messaggio in coda e apre una connessione TCP sulla porta 25 verso un server SMTP in esecuzione sul mail server dell'utente B.
4. Dopo un SMTP-handshake, il client SMTP invia il messaggio dell'utente A sulla connessione TCP instaurata.
5. Presso il mail server dell'utente B, il lato server di SMTP riceve il messaggio e lo posiziona nella casella postale.
6. Quando ritenuto opportuno, l'utente B potrà aprire e leggere la mail invocando il proprio user agent che userà il relativo protocollo di lettura. Ad oggi è spesso usato IMAP, altrimenti sono diffuse interfacce user agent basate su HTTP.

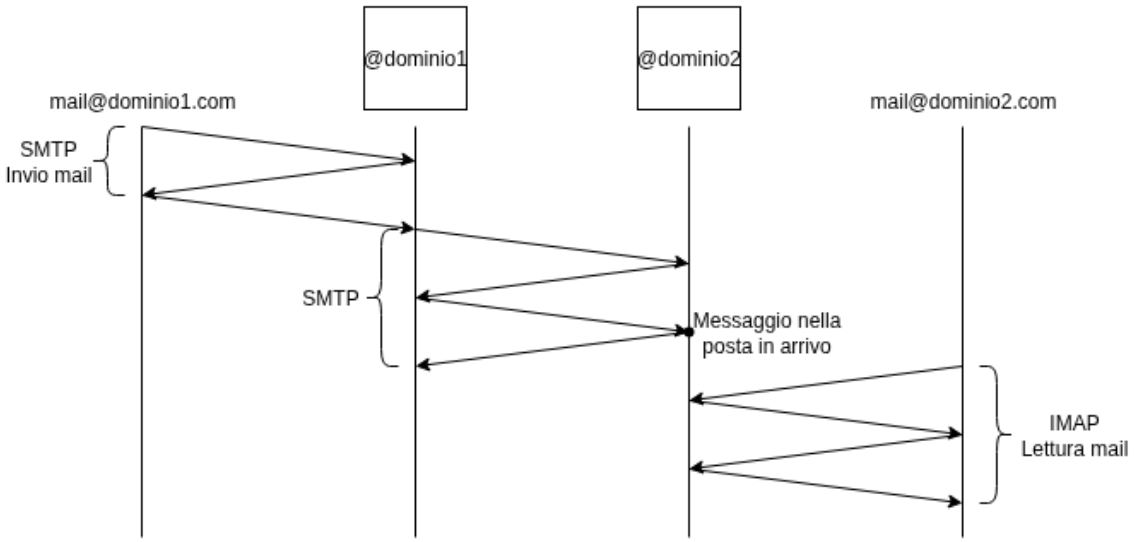


Figura 2.5: Invio e ricezione posta elettronica

Capitolo 3

Livello trasporto

3.1 Scopi e servizi

3.2 Protocollo TCP

3-way handshake, controllo di flusso, ritrasmissione, controllo di congestione

3.3 Protocollo UDP

Capitolo 4

Livello network

4.1 Protocollo IP

Formato del pacchetto IP, Indirizzi IP, spazio di indirizzamento, risoluzione degli indirizzi (ARP, RARP), CIDR

4.2 Protocollo di configurazione DHCP

4.3 Protocollo ICMP

4.4 Protocolli di routing

Distance vector, link state routing

4.5 IPv6

4.6 Network Address Translation - NAT

Capitolo 5

Livello data-link

5.1 Framing

5.2 Accesso al canale condiviso

5.3 Sottolivello MAC

Protocolli Aloha, CSMA, CSMA/CD, CSMA/CA

5.4 Bridge, switch e LAN

5.5 Wireless LAN